

DataFlix Project Status Report

Based on the original **Data_Science_Project_Proposal.pdf** and the development milestones, here is the current engineering summary for the DataFlix recommender pipeline.

✓ What We Have Successfully Done (100% Complete)

1. Data Pipeline & Cross-Domain Alignment

- **Data Loading:** Configured all local datasets (Netflix rating/movies, MovieLens ratings/movies, IMDb/TMDb files) cleanly with full offline access.
- **Cross-Domain Bridge (TF-IDF):** Implemented an advanced TF-IDF Title + Year fuzzy matching algorithm to natively map Netflix's non-standard internal IDs directly to MovieLens and TMDb IDs.
- **Sparse Feature Filtering:** Fully dropping unreliable users (<5 ratings) and movies (<10 ratings). Standardized user bias by applying per-user mean centering.

2. Feature Engineering & Deep Encoding (SBERT)

- **Dense Text Embeddings:** Ran `all-MiniLM-L6-v2` SBERT offline directly on the local TMDb `overview` text plots we aligned.
- **Meta-data Extraction:** Extracted and normalized one-hot encoded genres, release years, semantic popularity integers, runtimes, and original languages directly into a dense PyTorch feature representation.

3. Advanced Model Architecture (Hybrid + Dual Optimization)

- **Matrix Factorization (MF):** We developed the base Alternating Least Squares (ALS) and explicit MSE optimization paths.
- **BPR Ranking Optimization (Path B):** Built out an advanced implicit learning framework with popularity-proportional negative sampling.
- **Multi-Head Self-Attention:** Engineered an interaction fusion architecture that combines user-rating history embeddings and direct movie features into scalable state vectors.

4. End-to-End Evaluation Design

- Built an extensive evaluation script covering `RMSE` , `NDCG` , `MRR` , `Precision@K` , `Recall@K` , and `Coverage` .
- Configured 6 exact baselines as requested (Global Mean, Bias-Only, User-KNN, Vanilla MF, SVD++, NeuMF) matching standard academic literature.
- Automated ablation testing protocols and zero-shot (Cold-Start) testing scripts specifically targeting the Netflix user base.

5. Documentation & Delivery

- Completely cleaned up the **GitHub repository** structure, creating strict `.gitignore` configurations to prevent massive `.csv/.tsv` datasets from crashing git uploads.
- Generated a strictly formatted PDF summarizing the **Dataset Schema & Data Leakage Review**.

What is Remaining (Local Execution)

Because the pipeline is computationally heavy, the final training and evaluation steps must be executed natively on the local machine using the available CPU/GPU hardware.

1. Run the Full Model Evaluation (Local Execution)

Open the terminal in the VS Code project and manually run: ```bash python scripts/run_evaluate.py ``` (This triggers the training loop across the ALS, Baseline, and Hybrid Models to generate the final numeric values for the evaluation matrix.)

2. Generate the LaTeX Academic Table

Once `run_evaluate.py` outputs the final metrics (`results/metrics.csv`), that data needs to be manually transcribed into a LaTeX Table format for the university project report.

3. Project Final Report & Presentation

The core code, models, and data architecture are finished. The final manual steps are: *

- Moving the generated graphs from the `results/` folder into the Word/LaTeX academic document.
- * Summarizing the methodology based on the generated analysis PDFs for the final text submission.